

DOKUMEN TANYA JAWAB ASESOR

Skema Sertifikasi Programmer (SKKNI J.620100)

Persiapan Uji Lisan — Project NexaPOS (Sistem Point of Sale)

Laravel 11 (PHP 8.2/8.3) · MySQL 8 · Livewire · Redis

Disusun oleh: Muhammad Mizan Al Mujadid

Universitas Komputer Indonesia — Informatika Engineering

Pendahuluan

Dokumen ini berisi kumpulan pertanyaan yang berpotensi diajukan oleh asesor pada sesi uji lisan (wawancara) untuk setiap unit kompetensi skema Programmer (SKKNI J.620100), beserta jawaban yang merujuk pada implementasi nyata di project NexaPOS. Tujuannya adalah membantu memastikan pemahaman konsep di balik setiap bukti kompetensi, bukan hanya kemampuan menunjukkan kode jadi.

Disarankan untuk memahami alasan di balik tiap jawaban (bukan menghafal kata per kata), karena asesor dapat mengembangkan pertanyaan lanjutan berdasarkan jawaban yang diberikan.

1. J.620100.001.01 — Menganalisis Tools

P1: Mengapa Anda memilih Laravel dibanding framework PHP lain seperti CodeIgniter atau Symfony untuk project ini?

J: Laravel dipilih karena ekosistem lengkap (Eloquent ORM, Artisan, Livewire) yang mempercepat pengembangan dan mendukung pola OOP secara native. Sudah saya bandingkan dengan CodeIgniter (kurang terstruktur untuk OOP kompleks) dan Symfony (learning curve lebih tinggi untuk scope project ini) dalam dokumen analisis tools.

P2: Bagaimana Anda menguji coba tools sebelum memutuskan menggunakannya di project?

J: Saya membuat proof of concept kecil untuk komponen kritis, misalnya mencoba library zxing-js untuk scan barcode dan dompdf untuk cetak struk sebelum diintegrasikan penuh, agar risiko ketidakcocokan terdeteksi sejak awal.

P3: Risiko apa yang teridentifikasi dari penggunaan tools yang Anda pilih?

J: Risiko utama pada library scan barcode berbasis browser adalah ketergantungan pada kualitas kamera device dan pencahayaan; mitigasinya saya sediakan fallback input barcode manual.

2. J.620100.002.01 — Menganalisis Skalabilitas Perangkat Lunak

P1: Bagaimana sistem Anda menangani banyak kasir yang menulis ke tabel stok secara bersamaan?

J: Saya menggunakan row locking (``lockForUpdate()``) pada transaksi MySQL untuk mencegah race condition saat dua kasir mengurangi stok produk yang sama secara bersamaan, dibungkus dalam ``DB::transaction()``.

P2: Bagaimana Anda memperkirakan kebutuhan hardware untuk sistem ini?

J: Saya mengestimasi volume transaksi (~50/menit saat jam ramai) dan volume data historis (~100.000 baris transaksi), lalu menetapkan spesifikasi server minimum 2 vCPU/4GB RAM berdasarkan estimasi beban tersebut.

P3: Bagaimana jika sistem ini nantinya harus diperluas ke multi-cabang?

J: Scope v1.0 didesain single-store, tapi struktur data (mis. kolom `store_id`) sudah dipertimbangkan agar bisa ditambahkan, dan strategi sinkronisasi antar cabang dianalisis sebagai fase pengembangan berikutnya.

3. J.620100.003.01 — Melakukan Identifikasi Library, Komponen atau Framework yang Diperlukan

P1: Apa keuntungan menggunakan zxing-js dibanding membangun sendiri modul decode barcode?

J: Zxing-js sudah teruji luas, mendukung banyak format barcode/QR, dan dipelihara komunitas aktif — membangun sendiri akan memakan waktu jauh lebih lama dan rawan bug decoding.

P2: Bagaimana Anda merancang integrasi library ke dalam sistem?

J: Logic scan diisolasi ke komponen Livewire terpisah (``ScanProduct``) yang berkomunikasi via event ke ``TransactionCart``, sehingga jika library diganti di kemudian hari dampaknya terbatas pada satu komponen.

P3: Limitasi apa yang Anda temukan dari library yang dipakai?

J: Akurasi scan menurun pada pencahayaan buruk, dan dompdf tidak mendukung print langsung ke thermal printer sehingga perlu driver tambahan.

4. J.620100.006.01 — Merancang User Experience

P1: Bagaimana Anda menentukan batas maksimal aksi untuk alur transaksi?

J: Saya memetakan alur transaksi normal (scan-konfirmasi-bayar-cetak) dan menetapkan 4 aksi sebagai batas atas, lalu merancang layout agar tombol-tombol penting selalu terlihat tanpa perlu scroll.

P2: Bagaimana skenario ketika barcode gagal terbaca?

J: Saya sediakan skenario alternatif input manual, dengan tambahan aksi (cari produk, pilih) tapi tetap dijaga di bawah batas atas yang ditetapkan agar tidak memperlambat kasir secara signifikan.

5. J.620100.017.02 — Mengimplementasikan Pemrograman Terstruktur

P1: Kenapa Anda membuat fungsi-fungsi terpisah seperti `hitungDiskon()` sebelum masuk ke OOP penuh?

J: Untuk menunjukkan pemahaman pemrograman terstruktur dasar sebelum difaktorkan menjadi Service class — pendekatan bertahap ini juga memudahkan pengujian unit kecil sebelum digabung ke struktur OOP yang lebih kompleks.

P2: Bagaimana Anda menangani validasi input pada program?

J: Menggunakan struktur kontrol percabangan (if/else atau match expression) untuk memvalidasi qty, stok, dan metode pembayaran sebelum transaksi diproses.

6. J.620100.018.02 — Mengimplementasikan Pemrograman Berorientasi Objek

P1: Jelaskan penerapan inheritance dan polymorphism pada modul payment Anda.

J: `abstract class Payment` diturunkan menjadi `CashPayment`, `QrisPayment`, `CardPayment`. Saat `TransactionService` memanggil `\$payment->process()`, method yang dieksekusi berbeda tergantung objek konkretnya tanpa perlu tahu jenisnya — itu polymorphism.

P2: PHP tidak mendukung method overloading native, bagaimana Anda menunjukkan konsep ini?

J: Saya menggunakan pendekatan named constructor, seperti `Product::fromSingleItem()` dan `Product::fromBundle()`, sebagai representasi variasi cara pembuatan objek dengan parameter berbeda.

P3: Kenapa Anda memilih interface untuk PaymentGateway?

J: Interface memastikan semua class payment punya kontrak method yang sama (`process()`, `refund()`), sehingga TransactionService bergantung pada abstraksi bukan implementasi konkret — memudahkan penambahan metode pembayaran baru.

7. J.620100.020.02 — Menggunakan SQL

P1: Kenapa Anda menggunakan trigger untuk update stok, bukan hanya logic di aplikasi?

J: Trigger menjamin konsistensi data di level database — walau ada proses lain yang menulis langsung ke transaction_items, stok tetap otomatis ter-update, tidak bergantung sepenuhnya pada logic aplikasi.

P2: Jelaskan perbedaan penggunaan stored procedure dan function pada sistem Anda.

J: Stored procedure (`sp\_process\_transaction`) saya gunakan untuk operasi yang mengubah data (insert transaksi + update stok) dalam satu unit kerja atomic, sedangkan function (`fn\_calculate\_total`) saya gunakan untuk kalkulasi yang mengembalikan nilai tanpa mengubah data, sehingga bisa dipanggil dalam SELECT.

P3: Bagaimana Anda memastikan konsistensi data saat pembayaran gagal?

J: Seluruh proses transaksi dibungkus `DB::transaction()` / `START TRANSACTION` — jika pembayaran gagal di tengah proses, seluruh perubahan (termasuk pengurangan stok) di-rollback otomatis.

8. J.620100.021.02 — Menerapkan Akses Basis Data

P1: Bagaimana Anda mengamankan koneksi database?

J: Kredensial disimpan di file .env (tidak hardcoded), dan user MySQL aplikasi diberi hak akses terbatas tanpa izin DROP/ALTER, berbeda dari user admin database.

P2: Bagaimana Anda menguji performa query laporan?

J: Saya generate data dummy hingga 100.000 baris transaksi, lalu bandingkan waktu eksekusi query sebelum dan sesudah penambahan index menggunakan EXPLAIN untuk memverifikasi index terpakai.

9. J.620100.022.02 — Mengimplementasikan Algoritma Pemrograman

P1: Kenapa Anda membuat algoritma sorting/searching custom, padahal Eloquent punya orderBy dan where bawaan?

J: Untuk menunjukkan pemahaman kompleksitas algoritma secara eksplisit — saya implementasikan dan bandingkan linear search $O(n)$ vs pencarian berbasis index $O(\log n)$, serta custom sort dibandingkan orderBy bawaan, dengan pengukuran waktu eksekusi nyata sebagai bukti pemahaman.

P2: Bagaimana Anda menganalisis kompleksitas memori pada laporan dengan dataset besar?

J: Saya terapkan pagination pada laporan besar untuk membatasi jumlah data yang dimuat ke memori sekaligus, dibanding memuat seluruh dataset.

10. J.620100.024.02 — Melakukan Migrasi Ke Teknologi Baru

P1: Apa alasan migrasi dari polling ke Livewire real-time?

J: Polling menimbulkan latency dan beban server yang tidak perlu karena tiap kasir melakukan request berkala meski tidak ada perubahan data; Livewire + Laravel Echo memungkinkan update stok didorong secara real-time hanya saat terjadi perubahan, mengurangi beban server signifikan.

P2: Bagaimana Anda memastikan migrasi tidak merusak fitur yang sudah berjalan?

J: Saya jalankan test suite yang mencakup skenario update stok sebelum dan sesudah migrasi, serta melakukan rollout bertahap menggunakan feature flag.

11. J.620100.025.02 — Melakukan Debugging

P1: Ceritakan kasus debugging nyata yang Anda temukan.

J: Saya menemukan race condition saat dua kasir memproses transaksi untuk produk yang sama secara bersamaan, menyebabkan stok berpotensi menjadi negatif. Saya identifikasi lewat log QueryException, lalu perbaiki dengan row locking.

P2: Tools apa yang Anda gunakan untuk debugging?

J: Laravel Telescope untuk memonitor query dan request, serta Xdebug untuk step-through debugging pada logic kompleks di Service class.

12. J.620100.030.02 — Menerapkan Pemrograman Multimedia

P1: Kenapa memilih scan via kamera browser dibanding barcode scanner fisik?

J: Untuk fleksibilitas — tidak semua kasir punya hardware scanner fisik, dan browser modern sudah mendukung akses kamera langsung via getUserMedia API tanpa install aplikasi tambahan.

P2: Apa syarat teknis agar fitur scan kamera berfungsi?

J: Koneksi harus HTTPS (syarat wajib browser untuk izin akses kamera), dan browser harus mendukung getUserMedia API.

P3: Bagaimana Anda mengelola performa saat memproses frame video secara real-time?

J: Saya batasi interval pemrosesan deteksi barcode (misalnya setiap 200ms per frame) agar tidak membebani device secara berlebihan dibanding memproses tiap frame video yang masuk.

13. J.620100.032.01 — Menerapkan Code Review

P1: Bisa beri contoh temuan saat code review?

J: Saya menemukan TransactionController awal menangani terlalu banyak logic bisnis (validasi, hitung total, update stok) sekaligus, melanggar prinsip single responsibility — saya refactor logic itu ke TransactionService.

P2: Kapan Anda membuat pengecualian terhadap coding guideline?

J: Pada file migration yang berisi raw SQL untuk trigger dan stored procedure, saya kecualikan dari standar PSR-12 karena itu SQL murni bukan PHP, dengan komentar penjelasan di migration.

14. J.620100.036.02 — Melaksanakan Pengujian Kode Program Secara Statis

P1: Bagaimana Anda menguji modul payment tanpa gateway asli?

J: Saya buat stub/mock untuk QrisPayment dan CardPayment yang mensimulasikan response sukses/gagal, sehingga pengujian tidak bergantung pada koneksi ke gateway eksternal.

P2: Bagaimana Anda melakukan stress test?

J: Saya simulasikan 50 transaksi/menit menggunakan tool load testing, mengukur response time dan error rate, hasilnya didokumentasikan di test report.

15. J.620100.044.01 — Menerapkan Alert Notification Jika Aplikasi Bermasalah

P1: Bagaimana mekanisme notifikasi stok menipis bekerja?

J: Trigger database mendeteksi saat stok di bawah ambang minimum, lalu sistem mengirim notifikasi ke dashboard admin melalui Laravel Notification, dengan throttling maksimal 1x per produk per jam agar tidak spam.

P2: Bagaimana sistem tetap berjalan saat printer struk error?

J: Transaksi tetap tersimpan dengan status 'paid' di database meski cetak gagal, kasir bisa retry cetak ulang tanpa mengulang transaksi — ini prinsip graceful failure.

16. J.620100.045.01 — Melakukan Pemantauan Resource yang Digunakan Aplikasi

P1: Resource apa saja yang Anda monitor dan kenapa?

J: Koneksi database, memory server, response time API, dan jumlah job di queue — karena ini indikator kritikal yang jika melebihi ambang batas (misal memory >80%) bisa menyebabkan sistem tidak responsif.

P2: Bagaimana Anda memvisualisasikan data monitoring?

J: Menggunakan Laravel Telescope untuk request/query, ditambah dashboard custom dengan grafik line untuk response time dan gauge untuk resource usage, disertai garis ambang batas.

17. J.620100.047.01 — Melakukan Pembaharuan Perangkat Lunak

P1: Bagaimana Anda mengelola rilis fitur baru tanpa mengganggu sistem yang sudah berjalan?

J: Saya menggunakan feature flag untuk rollout bertahap (misalnya fitur multi-payment), serta mencatat changelog dan mengikuti semantic versioning agar perubahan terlacak jelas.

P2: Apa perbedaan versi v1.0 dan v1.1 pada sistem Anda?

J: v1.0 hanya mendukung pembayaran cash, v1.1 menambahkan QRIS, kartu, dan split bill — perbedaan ini didokumentasikan di CHANGELOG.md beserta migration database tambahan yang diperlukan.